UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS

INFORME DEL PROYECTO DE SISTEMAS OPERATIVOS

Algoritmos de reemplazo de páginas

CURSO: Sistemas Operativos

DOCENTE: Ing. Emilio Palomino Olivera

INTEGRANTES:

- Angel Ismael Alvarez Catunta — 221945
- [Nombre y Apellidos del Integrante 2] — [Código 2]
- [Nombre y Apellidos del Integrante 3] — [Código 3]
- [Nombre y Apellidos del Integrante 4] — [Código 4]
- [Nombre y Apellidos del Integrante 5] — [Código 5]
- [Nombre y Apellidos del Integrante 6] — [Código 6]

CUSCO - PERÚ

Semestre 2026 - II

INTRODUCCIÓN

Descripción y evolución del tema

La **memoria virtual** permite ejecutar procesos más grandes que la memoria física manteniendo en RAM solo las páginas en uso; apareció con el computador Atlas (Kilburn et al., 1962) y hoy se implementa mediante **paginación por demanda** (Silberschatz et al., 2018). Cuando una página referenciada no está en memoria ocurre un **fallo de página** y, si no hay marcos libres, un **algoritmo de reemplazo** elige la página víctima. Desde el algoritmo óptimo de Belady (1966) hasta WSClock (Carr & Hennessy, 1981), estos algoritmos buscan reducir los accesos al disco.

Problema

Cada fallo de página requiere una operación de E/S miles de veces más lenta que un acceso a RAM: un fallo cada 1 000 accesos basta para que la memoria parezca unas 40 veces más lenta (Silberschatz et al., 2018). Un mal algoritmo de reemplazo aumenta los fallos y puede llevar a la **hiperpaginación** (Denning, 1968).

Propuesta de solución

Se implementan en **lenguaje C para Linux** 12 algoritmos de reemplazo y se comparan bajo las mismas condiciones: **3 marcos** y una **cadena fija de 30 referencias** con un patrón de lecturas y escrituras. El simulador muestra la traza paso a paso y cuenta fallos, aciertos y escrituras a disco.

ÍNDICE

INTRODUCCIÓN **1**
Descripción y evolución del tema 1
Problema 1
Propuesta de solución 1
CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA Y METODOLOGÍA **3**
1.1 Planteamiento del problema 3
1.1.1 Descripción del problema 3
1.1.2 Identificación del problema 3
1.2 Justificación 3
1.3 Limitaciones 3
1.4 Delimitaciones 4
1.5 Metodología 4
1.5.1 Cronograma 4
CAPÍTULO II: MARCO TEÓRICO Y MODELADO DE LOS ALGORITMOS **5**
2.1 Antecedentes internacionales 5
2.2 Antecedentes nacionales 5
2.3 Temas relacionados al proyecto con modelado de cada algoritmo 5
2.3.1 Condiciones del experimento 5
2.3.2 Algoritmo 1: FIFO (First-In, First-Out) 6
2.3.3 Algoritmo 2: Óptimo (OPT / MIN) 7
2.3.4 Algoritmo 3: LRU (Least Recently Used) 9
2.3.5 Algoritmo 4: Reloj o Segunda Oportunidad (Clock) 10
2.3.6 Algoritmo 5: NRU (Not Recently Used) 11
2.3.7 Algoritmo 6: LFU (Least Frequently Used) 13
2.3.8 Algoritmo 7: MFU (Most Frequently Used) 14
2.3.9 Algoritmo 8: Reloj Mejorado (Enhanced Second Chance) 16
2.3.10 Algoritmo 9: Envejecimiento (Aging) 17
2.3.11 Algoritmo 10: Conjunto de Trabajo (Working Set) 19
2.3.12 Algoritmo 11: WSClock (Working Set Clock) 21
2.3.13 Algoritmo 12: Reemplazo Aleatorio (Random) 23
2.4 Comparación de resultados 24
CONCLUSIONES **26**
RECOMENDACIONES 26
BIBLIOGRAFÍA 26
ANEXO: COMPILACIÓN Y EJECUCIÓN EN LINUX 27

CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA Y METODOLOGÍA

1.1 Planteamiento del problema

1.1.1 Descripción del problema

El rendimiento de la memoria virtual depende de cuántos fallos de página se producen. El algoritmo óptimo los minimiza, pero necesita conocer el futuro; LRU se le acerca, pero es costoso de implementar en hardware. Las aproximaciones (Reloj, NRU, Aging, WSClock) equilibran complejidad y tasa de aciertos (Tanenbaum & Bos, 2015), y para elegir entre ellas se necesitan datos comparables.

1.1.2 Identificación del problema

- **Disparidad memoria–disco:** un fallo cuesta miles de veces más que un acierto.
- **Pocos marcos** frente al número de páginas del proceso.
- **El futuro es desconocido:** el algoritmo óptimo no es realizable.
- **Páginas modificadas:** expulsarlas obliga a escribirlas en disco.
- **Anomalía de Belády:** en algunos algoritmos, más marcos producen más fallos (Belady et al., 1969).

1.2 Justificación

El proyecto lleva a la práctica los conceptos de gestión de memoria del curso mediante código C real y medible, una competencia necesaria para optimizar sistemas donde la política de reemplazo determina el rendimiento (núcleos, bases de datos, cachés).

1.3 Limitaciones

- Simulación en espacio de usuario: no se modifica el núcleo ni se miden tiempos reales de disco.
- Una única cadena de 30 referencias: los resultados no se generalizan a cualquier carga.
- Un solo proceso con 3 marcos fijos y tiempo virtual (1 referencia = 1 tic).
- El resultado del algoritmo Aleatorio depende de la semilla (42).

1.4 Delimitaciones

- **Conceptual:** 12 algoritmos de reemplazo local de páginas.
- **Temporal:** Semestre 2026-II.
- **Hardware simulado:** 3 marcos de página.
- **Plataforma:** Linux (Ubuntu en WSL 2), gcc con C11 (`-Wall -Wextra -pedantic -O2`) y GNU Make.

1.5 Metodología

Enfoque **cuantitativo y experimental por simulación**: la variable independiente es el algoritmo; las dependientes, los fallos, aciertos, tasa de fallos y escrituras a disco; y se controlan la cadena, el patrón de lecturas/escrituras y el número de marcos. Los 12 algoritmos comparten una estructura modular (un archivo `.c` por algoritmo) y sus resultados se validaron con una implementación de referencia independiente y 67 pruebas automáticas (`make validar`), todas superadas.

1.5.1 Cronograma

Tabla 1

Cronograma del proyecto (formato de hoja de tareas de MS Project)

ID	Fase	Tarea	Días	Recursos	Pred.	Sem.
1	Planificación	Definir alcance, cadena de referencias y parámetros	2	Integrantes 1–6	—	1
2	Planificación	Reparto de algoritmos y creación del repositorio	1	Integrante 1	1	1
3	Marco teórico	Revisión bibliográfica de memoria virtual	4	Integrantes 2 y 3	2	1–2
4	Marco teórico	Búsqueda y síntesis de antecedentes	3	Integrantes 4 y 5	3	2
5	Implementación en C	Entorno Linux y estructura modular común	2	Integrante 1	2	1
6	Implementación en C	FIFO y Óptimo	4	Integrante 1	5	2
7	Implementación en C	LRU y Reloj	4	Integrante 2	5	2
8	Implementación en C	NRU y LFU	4	Integrante 3	5	2
9	Implementación en C	MFU y Reloj Mejorado	4	Integrante 4	5	2
10	Implementación en C	Aging y Working Set	4	Integrante 5	5	2
11	Implementación en C	WSClock y Aleatorio	4	Integrante 6	5	2
12	Simulación y pruebas	Validación formal del simulador	3	Integrantes 1 y 5	6–11	3
13	Simulación y pruebas	Generación de trazas, CSV y gráficas	2	Integrante 6	12	3
14	Redacción y validación	Redacción de Introducción y Capítulo I	3	Integrantes 2 y 3	4	3
15	Redacción y validación	Redacción del Capítulo II (cada integrante, sus 2 algoritmos)	5	Integrantes 1–6	13	4
16	Redacción y validación	Revisión integral, bibliografía APA 7 y entrega	2	Integrantes 1–6	14, 15	4

Nota. Integrante 1: Angel Ismael Alvarez Catunta; los integrantes 2 a 6 figuran en la carátula. Duración total: 21 días hábiles.

CAPÍTULO II: MARCO TEÓRICO Y MODELADO DE LOS ALGORITMOS

2.1 Antecedentes internacionales

Belady, L. A. (1966). *A study of replacement algorithms for a virtual-storage computer*. Comparó algoritmos de reemplazo mediante simulación de trazas y definió el algoritmo óptimo (MIN), que expulsa la página cuyo próximo uso está más lejos. **Relación con el proyecto:** Es el Óptimo que este proyecto usa como cota mínima (15 fallos).

Denning, P. J. (1968). *The working set model for program behavior*. Definió el conjunto de trabajo $W(t, \tau)$ y explicó la hiperpaginación como resultado de asignar menos marcos que los que exige la localidad del programa. **Relación con el proyecto:** Fundamenta los algoritmos Working Set y WSClock implementados ($\tau = 4$).

2.2 Antecedentes nacionales

Ruiz L., E., & Raffo L., E. (2003). *Simulación de la memoria cache*. Industrial Data, 6(2), 48–55. Universidad Nacional Mayor de San Marcos. Simularon en C++ una memoria caché con reemplazo FIFO para estimar el porcentaje de aciertos y el tiempo medio de acceso, y hallaron un tamaño óptimo de caché a partir del cual los aciertos no mejoran. **Relación con el proyecto:** Comparte el enfoque de simulación en C/C++, el uso de FIFO y la métrica de aciertos.

[COMPLETAR – Antecedente nacional 2]: agregar una tesis o artículo peruano verificado (repositorio de la UNSAAC, ALICIA/CONCYTEC o RENATI) y su referencia en la Bibliografía.

2.3 Temas relacionados al proyecto con modelado de cada algoritmo

2.3.1 Condiciones del experimento

Todos los algoritmos se ejecutan con **3 marcos inicialmente vacíos** (las primeras cargas cuentan como fallo) sobre la cadena oficial de la Tabla 2, que tiene 7 páginas distintas (mínimo 7 fallos). Si hay un marco libre se usa el de menor índice. En cada traza, las filas M1–M3 muestran el contenido de los marcos **después** de cada referencia; la página cargada aparece **sombreada**, y la fila Res indica fallo (F) o acierto (H). Cada módulo se compila con los archivos comunes del proyecto (`make`) y se ejecuta con `./simulador -a N`.

Tabla 2

Cadena oficial de 30 referencias (R = lectura, W = escritura)

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Página	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R

2.3.2 Algoritmo 1: FIFO (First-In, First-Out)

Responsable: Integrante 1 – Angel Ismael Alvarez Catunta.

Principio de operación. Expulsa la página que lleva **más tiempo en memoria**, como una cola; un acierto no cambia el orden. Se implementa con un puntero circular al marco más antiguo. **Ventaja:** muy simple y de costo O(1). **Desventaja:** ignora el uso de las páginas. **Anomalía de Belády: sí** (9 → 10 fallos con la cadena de Belády). (Silberschatz et al. (2018)).

Tabla 3

Traza de FIFO con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	0	0	0	0	**7**	7	7	**3**	3	3	**0**	0	0	**2**	2	2	**1**
M2	-	**0**	0	0	0	**3**	3	3	**2**	2	2	2	2	**1**	1	1	1	1	**0**	0	0	**5**	5	5	**1**	1	1	**3**	3	3
M3	-	-	**1**	1	1	1	**0**	0	0	**3**	3	3	3	3	**2**	2	2	2	2	**1**	1	1	**2**	2	2	**4**	4	4	**0**	0
Res	**F**	**F**	**F**	**F**	H	**F**	**F**	**F**	**F**	**F**	**F**	H	H	**F**	**F**	H	H	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**
Sale				7		0	1	2	3	0	4			2	3			0	1	2	7	0	1	3	5	2	0	1	4	2

Resultado: 25 fallos, 5 aciertos, tasa de fallos 83.33 % y 9 escrituras a disco. Expulsa en el paso 6 la página 0, que se necesita en el paso 7.

Código fuente en C: `src/algoritmos/fifo.c` (ejecución: `./simulador -a 1`)

```
/*
 * fifo.c - Algoritmo 1: FIFO (First-In, First-Out)
 * Responsable: Integrante 1
 *
 * Idea: se expulsa la página que lleva más tiempo en memoria.
 * Como los marcos vacíos se llenan en orden (M1, M2, M3) y cada reemplazo
 * ocurre en el marco más antiguo, basta un puntero circular que indica
 * "el marco que llegó primero". Un acierto NO modifica la cola.
 *
 * Variables auxiliares mostradas: cola de llegada (más antigua primero).
 *
 * Este módulo es la PLANTILLA de referencia para los demás algoritmos.
 */
#include "../algoritmos.h"

/* Arma la cola de llegada empezando por el marco más antiguo */
static const char *cola_fifo(char *buf, size_t len, const int marcos[], int m, int puntero)
{
    int cola[MAX_MARCOS];
    int cant = 0;

    for (int k = 0; k < m; k++) {
        int idx = (puntero + k) % m;
```

```
        if (marcos[idx] != MARCO_VACIO)
            cola[cant++] = marcos[idx];
    }
    return fmt_arreglo(buf, len, cola, cant);
}

void simular_fifo(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int puntero = 0;            /* marco con la página más antigua */
    char buf[AUX_LEN];

    resultado_iniciar(r, "FIFO", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int marco, victima = -1;

        if (buscar_pagina(marcos, cfg->marcos, pagina) >= 0) {
            /* Acierto: la cola no cambia */
            registrar_paso(r, cfg, i, marcos, -1, -1, "cola=%s",
                           cola_fifo(buf, sizeof buf, marcos, cfg->marcos, puntero));
            continue;
        }

        /* Fallo de página */
        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = puntero;                      /* el más antiguo */
            victima = marcos[marco];
            puntero = (puntero + 1) % cfg->marcos;
        }
        marcos[marco] = pagina;

        registrar_paso(r, cfg, i, marcos, marco, victima, "cola=%s",
                       cola_fifo(buf, sizeof buf, marcos, cfg->marcos, puntero));
    }
}
```

2.3.3 Algoritmo 2: Óptimo (OPT / MIN)

Responsable: Integrante 1 – Angel Ismael Alvarez Catunta.

Principio de operación. Expulsa la página cuyo **próximo uso está más lejos** en el futuro. Da el mínimo de fallos posible, pero no es realizable porque requiere conocer la cadena completa; se usa como cota de comparación. **Ventaja:** mínimo de fallos. **Desventaja:** no realizable en un sistema real. **Anomalía de Belády: no** (algoritmo de pila). (Belady (1966)).

Tabla 4

Traza de Óptimo con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	2	2	2	2	2	2	2	2	2	2	**7**	7	7	**3**	**5**	**2**	2	2	2	2	**3**	3	**1**
M2	-	**0**	0	0	0	0	0	**4**	4	4	**0**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
M3	-	-	**1**	1	1	**3**	3	3	3	3	3	3	3	**1**	1	1	1	1	1	1	1	1	1	1	1	**4**	4	4	4	4
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	H	H	**F**	H	H	**F**	H	H	H	**F**	H	H	**F**	**F**	**F**	H	H	**F**	H	**F**	H	**F**
Sale				7		1		0			4			3				2			7	3	5			1		2		3

Resultado: 15 fallos, 15 aciertos, tasa de fallos 50.00 % y 6 escrituras a disco. En el paso 18 expulsa la página 2, que no vuelve a usarse hasta el paso 23.

Código fuente en C: `src/algoritmos/optimo.c` (ejecución: `./simulador -a 2`)

```c
/*
 * optimo.c - Algoritmo 2: Óptimo (OPT / MIN de Belády)
 * Responsable: Integrante 1
 *
 * Idea: se expulsa la página cuyo PRÓXIMO uso está más lejos en el futuro
 * (o que no se volverá a usar). Requiere conocer toda la cadena por
 * adelantado, por eso no es implementable en un SO real; sirve como
 * cota mínima de fallos para comparar a los demás algoritmos.
 *
 * Empate (varias páginas que no se vuelven a usar): menor índice de marco.
 * Variables auxiliares: paso del próximo uso de cada marco ("--" = nunca).
 *   En un reemplazo se muestran los valores con los que se decidió.
 */
#include "../algoritmos.h"

#define NUNCA 1000000   /* distancia "infinita": la página no se vuelve a usar */

/* Paso (0-based) del próximo uso de 'pagina' después de 'desde', o NUNCA */
static int proximo_uso(const Config *cfg, int pagina, int desde)
{
    for (int j = desde + 1; j < cfg->n; j++)
        if (cfg->refs[j] == pagina)
            return j;
    return NUNCA;
}

/* Texto "prox=[5,14,--]" con pasos en base 1 */
static const char *fmt_proximos(char *buf, size_t len, const Config *cfg,
                                const int marcos[], int i)
{
    size_t u = (size_t)snprintf(buf, len, "prox=[");
    for (int k = 0; k < cfg->marcos && u < len; k++) {
        const char *sep = (k + 1 < cfg->marcos) ? "," : "]";
        int prox = marcos[k] == MARCO_VACIO ? NUNCA : proximo_uso(cfg, marcos[k], i);
        if (marcos[k] == MARCO_VACIO)
            u += (size_t)snprintf(buf + u, len - u, "-%s", sep);
        else if (prox == NUNCA)
            u += (size_t)snprintf(buf + u, len - u, "--%s", sep);
        else
            u += (size_t)snprintf(buf + u, len - u, "%d%s", prox + 1, sep);
    }
    return buf;
}

void simular_optimo(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    char buf[AUX_LEN];

    resultado_iniciar(r, "Optimo (OPT)", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int marco, victima = -1;

        if (buscar_pagina(marcos, cfg->marcos, pagina) >= 0) {
            registrar_paso(r, cfg, i, marcos, -1, -1, "%s",
                           fmt_proximos(buf, sizeof buf, cfg, marcos, i));
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            int mas_lejano = -1;
            /* Decidir con el estado ANTES del reemplazo */
```

```
            fmt_proximos(buf, sizeof buf, cfg, marcos, i);
            for (int k = 0; k < cfg->marcos; k++) {
                int prox = proximo_uso(cfg, marcos[k], i);
                if (prox > mas_lejano) {        /* '>' estricto: empate -> menor índice */
                    mas_lejano = prox;
                    marco = k;
                }
            }
            victima = marcos[marco];
            marcos[marco] = pagina;
            registrar_paso(r, cfg, i, marcos, marco, victima, "%s -> sale la mas lejana", buf);
            continue;
        }

        marcos[marco] = pagina;
        registrar_paso(r, cfg, i, marcos, marco, victima, "%s",
                       fmt_proximos(buf, sizeof buf, cfg, marcos, i));
    }
}
```

2.3.4 Algoritmo 3: LRU (Least Recently Used)

Responsable: Integrante 2 – [Nombre y Apellidos del Integrante 2].

Principio de operación. Expulsa la página que lleva **más tiempo sin usarse**, aprovechando la localidad temporal. Cada marco guarda el paso de su último uso y sale el menor. **Ventaja:** buena aproximación del Óptimo. **Desventaja:** su versión exacta es costosa en hardware. **Anomalía de Belády: no** (algoritmo de pila). (Silberschatz et al. (2018)).

Tabla 5

Traza de LRU con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	**1**	1	1	1	1	1	1	1	1	**2**	2	2	**4**	4	4	**0**	0
M2	-	**0**	0	0	0	0	0	0	0	**3**	3	3	3	3	3	**0**	0	0	0	0	0	**5**	5	5	**1**	1	1	**3**	3	3
M3	-	-	**1**	1	1	**3**	3	3	**2**	2	2	2	2	2	2	2	2	**7**	7	7	**3**	3	3	**0**	0	0	**2**	2	2	**1**
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	**F**	**F**	**F**	H	H	**F**	H	**F**	H	**F**	H	H	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**
Sale				7		1		2	3	0	4			0		3		2			7	0	1	3	5	2	0	1	4	2

Resultado: 22 fallos, 8 aciertos, tasa de fallos 73.33 % y 8 escrituras a disco. La víctima es siempre la página con el último uso más antiguo.

Código fuente en C: `src/algoritmos/lru.c` (ejecución: `./simulador -a 3`)

```
/*
 * lru.c - Algoritmo 3: LRU (Least Recently Used)
 * Responsable: Integrante 2
 *
 * Idea: se expulsa la página que lleva MÁS TIEMPO sin usarse.
 * Implementación con "contador/marca de tiempo" (Silberschatz): cada marco
 * guarda el paso de su último uso; la víctima es la marca más pequeña.
 * Es un algoritmo de pila: no sufre la anomalía de Belády.
 *
 * Variables auxiliares: ult = paso (base 1) del último uso de cada marco.
 */
#include "../algoritmos.h"

void simular_lru(const Config *cfg, Resultado *r)
```

```
{
    int marcos[MAX_MARCOS];
    int ultimo[MAX_MARCOS];     /* paso del último uso, base 1 */
    char buf[AUX_LEN];

    resultado_iniciar(r, "LRU", cfg);
    marcos_vaciar(marcos, cfg->marcos);
    marcos_vaciar(ultimo, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1;

        if (idx >= 0) {
            ultimo[idx] = i + 1;
            registrar_paso(r, cfg, i, marcos, -1, -1, "ult=%s",
                           fmt_arreglo(buf, sizeof buf, ultimo, cfg->marcos));
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = 0;
            for (int k = 1; k < cfg->marcos; k++)
                if (ultimo[k] < ultimo[marco])
                    marco = k;
            victima = marcos[marco];
        }
        marcos[marco] = pagina;
        ultimo[marco] = i + 1;
        registrar_paso(r, cfg, i, marcos, marco, victima, "ult=%s",
                       fmt_arreglo(buf, sizeof buf, ultimo, cfg->marcos));
    }
}
```

2.3.5 Algoritmo 4: Reloj o Segunda Oportunidad (Clock)

Responsable: Integrante 2 – [Nombre y Apellidos del Integrante 2].

Principio de operación. FIFO circular con **bit de referencia R**: si la página bajo el puntero tiene R = 1, se pone R = 0 y se avanza (segunda oportunidad); si R = 0, se expulsa. **Ventaja:** aproxima LRU con un solo bit. **Desventaja:** si todos tienen R = 1 se comporta como FIFO. **Anomalía de Belády: sí** (9 → 10). (Corbató (1969)).

Tabla 6

Traza de Reloj o Segunda Oportunidad con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	**4**	4	4	4	**3**	3	3	3	**0**	0	0	0	0	**3**	3	3	**0**	0	0	**2**	2	2	**1**
M2	-	**0**	0	0	0	0	0	0	**2**	2	2	2	2	**1**	1	1	1	**7**	7	7	7	**5**	5	5	**1**	1	1	**3**	3	3
M3	-	-	**1**	1	1	**3**	3	3	3	3	**0**	0	0	0	**2**	2	2	2	2	**1**	1	1	**2**	2	2	**4**	4	4	**0**	0
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	**F**	H	**F**	**F**	H	**F**	**F**	**F**	H	**F**	H	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**
Sale				7		1		2	0		3	4		2	0	3		1		2	0	7	1	3	5	2	0	1	4	2

Resultado: 24 fallos, 6 aciertos, tasa de fallos 80.00 % y 9 escrituras a disco. En el paso 18 los tres bits R valen 1 y el puntero da una vuelta completa.

Código fuente en C: `src/algoritmos/reloj.c` (ejecución: `./simulador -a 4`)

```c
/*
 * reloj.c - Algoritmo 4: Reloj / Segunda Oportunidad (Clock)
 * Responsable: Integrante 2
 *
 * Idea: FIFO circular con bit de referencia R. El puntero (manecilla)
 * recorre los marcos: si R = 1 se le da una "segunda oportunidad"
 * (R = 0 y avanza); si R = 0, esa página es la víctima.
 * Al cargar o en un acierto: R = 1. Tras reemplazar, el puntero avanza.
 * El puntero empieza en M1 y no se mueve mientras se llenan marcos libres.
 *
 * Variables auxiliares: bits R de cada marco y posición del puntero.
 */
#include "../algoritmos.h"

void simular_reloj(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int puntero = 0;
    char buf[AUX_LEN];

    resultado_iniciar(r, "Reloj (Clock)", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1, segundas = 0;

        if (idx >= 0) {
            bit_r[idx] = 1;
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s ptr=M%d",
                           fmt_arreglo(buf, sizeof buf, bit_r, cfg->marcos), puntero + 1);
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            while (bit_r[puntero] == 1) {       /* segunda oportunidad */
                bit_r[puntero] = 0;
                puntero = (puntero + 1) % cfg->marcos;
                segundas++;
            }
            marco = puntero;
            victima = marcos[marco];
            puntero = (puntero + 1) % cfg->marcos;
        }
        marcos[marco] = pagina;
        bit_r[marco] = 1;

        if (victima >= 0)
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s ptr=M%d (2das oport.: %d)",
                           fmt_arreglo(buf, sizeof buf, bit_r, cfg->marcos), puntero + 1, segundas);
        else
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s ptr=M%d",
                           fmt_arreglo(buf, sizeof buf, bit_r, cfg->marcos), puntero + 1);
    }
}
```

2.3.6 Algoritmo 5: NRU (Not Recently Used)

Responsable: Integrante 3 – [Nombre y Apellidos del Integrante 3].

Principio de operación. Clasifica las páginas por sus bits (R, M) en 4 clases y expulsa una de la **clase más baja** (no usada y limpia primero). Cada 5 tics una interrupción de reloj pone R = 0.

Empate: la más antigua. **Ventaja:** simple y evita escrituras. **Desventaja:** decisión gruesa (solo 4 clases). **Anomalía de Belády:** posible (no es de pila); no se observó. (Tanenbaum y Bos (2015)).

Tabla 7

Traza de NRU con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	**4**	4	4	4	**3**	3	**1**	1	1	1	1	1	1	**3**	3	3	3	**1**	**4**	4	4	**0**	0
M2	-	**0**	0	0	0	0	0	0	0	**3**	**0**	0	0	0	0	0	0	0	0	0	0	0	**2**	2	2	2	2	2	2	**1**
M3	-	-	**1**	1	1	**3**	3	3	**2**	2	2	2	2	2	2	2	2	**7**	7	7	7	**5**	5	**0**	0	0	0	**3**	3	3
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	**F**	**F**	**F**	**F**	H	**F**	H	H	H	**F**	H	H	**F**	**F**	**F**	**F**	**F**	**F**	H	**F**	**F**	**F**
Sale				7		1		2	3	0	3	4		3				2			1	7	0	5	3	1		0	4	2

Resultado: 21 fallos, 9 aciertos, tasa de fallos 70.00 % y 7 escrituras a disco. En el paso 21 el reloj acaba de limpiar R y sale la página 1, de clase 0.

Código fuente en C: `src/algoritmos/nru.c` (ejecución: `./simulador -a 5`)

```c
/*
 * nru.c - Algoritmo 5: NRU (Not Recently Used)
 * Responsable: Integrante 3
 *
 * Idea (Tanenbaum 3.4.2): cada página tiene bits R (referenciada) y
 * M (modificada). Una interrupción de reloj cada PERIODO_RELOJ tics pone
 * todos los R en 0. Las páginas se agrupan en 4 clases:
 *     clase 0 = (R=0,M=0)  no usada, limpia     <- mejor víctima
 *     clase 1 = (R=0,M=1)  no usada, sucia
 *     clase 2 = (R=1,M=0)  usada, limpia
 *     clase 3 = (R=1,M=1)  usada, sucia
 * Se expulsa una página de la clase más baja; empate: la más antigua (FIFO).
 * A diferencia del Reloj Mejorado, revisa TODA la tabla (sin puntero).
 *
 * Variables auxiliares: bits R y M, y aviso cuando el reloj limpia R.
 */
#include "../algoritmos.h"

void simular_nru(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int bit_m[MAX_MARCOS] = {0};
    int llegada[MAX_MARCOS] = {0};
    char br[AUX_LEN / 3], bm[AUX_LEN / 3];

    resultado_iniciar(r, "NRU", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int w = es_escritura(cfg, i);
        int reset = (i > 0 && i % PERIODO_RELOJ == 0);
        int idx, marco, victima = -1, clase_victima = -1;

        /* Interrupción de reloj: limpia los bits R al inicio del tic */
        if (reset)
            for (int k = 0; k < cfg->marcos; k++)
                bit_r[k] = 0;

        idx = buscar_pagina(marcos, cfg->marcos, pagina);
        if (idx >= 0) {
            bit_r[idx] = 1;
            bit_m[idx] |= w;
```

```
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s M=%s%s",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           reset ? " [reloj: R<-0]" : "");
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            int mejor = 0;
            for (int k = 1; k < cfg->marcos; k++) {
                int ck = 2 * bit_r[k] + bit_m[k];
                int cm = 2 * bit_r[mejor] + bit_m[mejor];
                if (ck < cm || (ck == cm && llegada[k] < llegada[mejor]))
                    mejor = k;
            }
            marco = mejor;
            clase_victima = 2 * bit_r[marco] + bit_m[marco];
            victima = marcos[marco];
        }
        marcos[marco] = pagina;
        bit_r[marco] = 1;
        bit_m[marco] = w;
        llegada[marco] = i;

        if (victima >= 0)
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s clase=%d%s",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           clase_victima, reset ? " [reloj: R<-0]" : "");
        else
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s%s",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           reset ? " [reloj: R<-0]" : "");
    }
}
```

2.3.7 Algoritmo 6: LFU (Least Frequently Used)

Responsable: Integrante 3 – [Nombre y Apellidos del Integrante 3].

Principio de operación. Expulsa la página con **menor contador de usos** (el contador vuelve a 1 al cargar). Empate: la más antigua. **Ventaja:** protege las páginas muy usadas. **Desventaja:** una página antigua muy usada puede quedarse sin necesitarse. **Anomalía de Belády:** posible (se halló un caso: 9 → 11). (Silberschatz et al. (2018)).

Tabla 8

Traza de LFU con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	**4**	4	**3**	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
M2	-	**0**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
M3	-	-	**1**	1	1	**3**	3	3	**2**	2	2	2	2	**1**	**2**	2	**1**	**7**	7	**1**	1	**5**	**2**	2	**1**	**4**	**2**	2	2	**1**
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	**F**	**F**	H	H	H	**F**	**F**	H	**F**	**F**	H	**F**	H	**F**	**F**	H	**F**	**F**	**F**	H	H	**F**
Sale				7		1		2	3	4				2	1		2	1		7		1	5		2	1	4			2

Resultado: 19 fallos, 11 aciertos, tasa de fallos 63.33 % y 4 escrituras a disco. La página 0, la más usada y escrita, no es expulsada desde el paso 15.

Código fuente en C: `src/algoritmos/lfu.c` (ejecución: `./simulador -a 6`)

```c
/*
 * lfu.c - Algoritmo 6: LFU (Least Frequently Used)
 * Responsable: Integrante 3
 *
 * Idea: cada página en memoria lleva un contador de usos; se expulsa la
 * de MENOR contador. El contador se reinicia a 1 al cargar la página
 * (no se guarda historial de páginas que están fuera de memoria).
 * Empate: la que llegó primero (FIFO).
 *
 * Variables auxiliares: contador de frecuencia de cada marco.
 */
#include "../algoritmos.h"

void simular_lfu(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int frec[MAX_MARCOS] = {0};
    int llegada[MAX_MARCOS] = {0};
    char buf[AUX_LEN];

    resultado_iniciar(r, "LFU", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1;

        if (idx >= 0) {
            frec[idx]++;
            registrar_paso(r, cfg, i, marcos, -1, -1, "frec=%s",
                           fmt_arreglo(buf, sizeof buf, frec, cfg->marcos));
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = 0;
            for (int k = 1; k < cfg->marcos; k++)
                if (frec[k] < frec[marco] ||
                    (frec[k] == frec[marco] && llegada[k] < llegada[marco]))
                    marco = k;
            victima = marcos[marco];
        }
        marcos[marco] = pagina;
        frec[marco] = 1;
        llegada[marco] = i;
        registrar_paso(r, cfg, i, marcos, marco, victima, "frec=%s",
                       fmt_arreglo(buf, sizeof buf, frec, cfg->marcos));
    }
}
```

2.3.8 Algoritmo 7: MFU (Most Frequently Used)

Responsable: Integrante 4 – [Nombre y Apellidos del Integrante 4].

Principio de operación. Expulsa la página con **mayor contador de usos**, suponiendo que la de contador bajo acaba de llegar y aún se usará. Empate: la más antigua. **Ventaja:** simple.

Desventaja: expulsa las páginas más útiles. **Anomalía de Belády: sí** (9 → 10). (Silberschatz et al. (2018)).

Tabla 9

Traza de MFU con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	0	0	0	0	**7**	7	7	**3**	3	3	**0**	0	0	**2**	2	2	**1**
M2	-	**0**	0	0	0	**3**	3	3	**2**	2	2	2	2	**1**	1	1	1	1	**0**	0	0	**5**	5	5	**1**	1	1	**3**	3	3
M3	-	-	**1**	1	1	1	**0**	0	0	**3**	3	3	3	3	**2**	2	2	2	2	**1**	1	1	**2**	2	2	**4**	4	4	**0**	0
Res	**F**	**F**	**F**	**F**	H	**F**	**F**	**F**	**F**	**F**	**F**	H	H	**F**	**F**	H	H	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**
Sale				7		0	1	2	3	0	4			2	3			0	1	2	7	0	1	3	5	2	0	1	4	2

Resultado: 25 fallos, 5 aciertos, tasa de fallos 83.33 % y 9 escrituras a disco. Tras sacar las páginas más usadas, todas quedan con contador 1 y se comporta como FIFO.

Código fuente en C: `src/algoritmos/mfu.c` (ejecución: `./simulador -a 7`)

```c
/*
 * mfu.c - Algoritmo 7: MFU (Most Frequently Used)
 * Responsable: Integrante 4
 *
 * Idea: se expulsa la página con MAYOR contador de usos, bajo el
 * argumento de que una página muy usada "ya cumplió su función" y la de
 * contador bajo acaba de llegar y aún se usará. Mismo contador que LFU
 * (se reinicia a 1 al cargar). Empate: la que llegó primero (FIFO).
 *
 * Variables auxiliares: contador de frecuencia de cada marco.
 */
#include "../algoritmos.h"

void simular_mfu(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int frec[MAX_MARCOS] = {0};
    int llegada[MAX_MARCOS] = {0};
    char buf[AUX_LEN];

    resultado_iniciar(r, "MFU", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1;

        if (idx >= 0) {
            frec[idx]++;
            registrar_paso(r, cfg, i, marcos, -1, -1, "frec=%s",
                           fmt_arreglo(buf, sizeof buf, frec, cfg->marcos));
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = 0;
            for (int k = 1; k < cfg->marcos; k++)
                if (frec[k] > frec[marco] ||
                    (frec[k] == frec[marco] && llegada[k] < llegada[marco]))
                    marco = k;
            victima = marcos[marco];
        }
```

```
        marcos[marco] = pagina;
        frec[marco] = 1;
        llegada[marco] = i;
        registrar_paso(r, cfg, i, marcos, marco, victima, "frec=%s",
                       fmt_arreglo(buf, sizeof buf, frec, cfg->marcos));
    }
}
```

2.3.9 Algoritmo 8: Reloj Mejorado (Enhanced Second Chance)

Responsable: Integrante 4 – [Nombre y Apellidos del Integrante 4].

Principio de operación. Reloj con el par (R, M): primero busca (0,0) sin tocar bits; luego (0,1) poniendo R = 0 al pasar, y repite. Prefiere páginas **no usadas y limpias**. **Ventaja:** reduce escrituras a disco. **Desventaja:** hasta 4 vueltas del puntero. **Anomalía de Belády: sí** (9 → 10). (Silberschatz et al. (2018)).

Tabla 10

Traza de Reloj Mejorado con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	0	0	0	0	0	0	0	0	0	**2**	2	**1**	1	**2**	2	2	**1**
M2	-	**0**	0	0	0	0	0	0	0	**3**	3	3	3	**1**	1	1	1	**7**	7	7	**3**	3	3	3	3	**4**	4	**3**	3	3
M3	-	-	**1**	1	1	**3**	3	3	**2**	2	2	2	2	2	2	2	2	2	2	**1**	1	**5**	5	**0**	0	0	0	0	0	0
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	**F**	**F**	**F**	H	H	**F**	H	H	H	**F**	H	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	H	**F**
Sale				7		1		2	3	0	4			3				1		2	7	1	0	5	2	3	1	4		2

Resultado: 21 fallos, 9 aciertos, tasa de fallos 70.00 % y 6 escrituras a disco. Iguala a NRU en fallos (21) pero hace una escritura menos.

Código fuente en C: `src/algoritmos/reloj_mejorado.c` (ejecución: `./simulador -a 8`)

```
/*
 * reloj_mejorado.c - Algoritmo 8: Reloj Mejorado (Enhanced Second Chance)
 * Responsable: Integrante 4
 *
 * Idea (Silberschatz): Reloj que considera el par (R, M):
 *     (0,0) no usada, limpia  -> mejor víctima
 *     (0,1) no usada, sucia   -> hay que escribirla antes
 *     (1,0) usada, limpia
 *     (1,1) usada, sucia      -> peor víctima
 * Búsqueda con el puntero, desde su posición actual:
 *   Pasada 1: buscar (0,0) dando una vuelta SIN modificar bits.
 *   Pasada 2: buscar (0,1) poniendo R = 0 en cada marco recorrido.
 *   Si no hubo víctima, se repite (ahora todos tienen R = 0).
 * Tras reemplazar, el puntero queda en el marco siguiente.
 *
 * Variables auxiliares: bits R y M, puntero y pasada en que se halló la víctima.
 */
#include "../algoritmos.h"

void simular_reloj_mejorado(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int bit_m[MAX_MARCOS] = {0};
    int puntero = 0;
    char br[AUX_LEN / 3], bm[AUX_LEN / 3];
```

```
    resultado_iniciar(r, "Reloj Mejorado", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int w = es_escritura(cfg, i);
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1, pasada = 0;

        if (idx >= 0) {
            bit_r[idx] = 1;
            bit_m[idx] |= w;
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s M=%s ptr=M%d",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos), puntero + 1);
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = -1;
            while (marco < 0) {
                /* Pasada 1: (0,0) sin tocar bits */
                pasada++;
                for (int k = 0; k < cfg->marcos; k++) {
                    int c = (puntero + k) % cfg->marcos;
                    if (bit_r[c] == 0 && bit_m[c] == 0) {
                        marco = c;
                        break;
                    }
                }
                if (marco >= 0)
                    break;
                /* Pasada 2: (0,1) limpiando R al pasar */
                pasada++;
                for (int k = 0; k < cfg->marcos; k++) {
                    int c = (puntero + k) % cfg->marcos;
                    if (bit_r[c] == 0 && bit_m[c] == 1) {
                        marco = c;
                        break;
                    }
                    bit_r[c] = 0;
                }
            }
            victima = marcos[marco];
            puntero = (marco + 1) % cfg->marcos;
        }
        marcos[marco] = pagina;
        bit_r[marco] = 1;
        bit_m[marco] = w;

        if (victima >= 0)
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s ptr=M%d (pasada %d)",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos), puntero + 1, pasada);
        else
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s ptr=M%d",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos), puntero + 1);
    }
}
```

2.3.10 Algoritmo 9: Envejecimiento (Aging)

Responsable: Integrante 5 – [Nombre y Apellidos del Integrante 5].

Principio de operación. Cada marco tiene un **registro de 8 bits**; en cada tic se desplaza a la derecha y entra R por la izquierda. Sale la página con el registro menor. **Ventaja:** aproxima LRU

con pocos bits. **Desventaja:** solo recuerda los últimos 8 tics. **Anomalía de Belády:** posible (no es de pila); no se observó. (Tanenbaum y Bos (2015)).

Tabla 11

Traza de Envejecimiento con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	**1**	1	1	1	1	1	1	1	1	**2**	2	2	**4**	4	4	**0**	0
M2	-	**0**	0	0	0	0	0	0	0	**3**	3	3	3	3	3	**0**	0	0	0	0	0	**5**	5	5	**1**	1	1	**3**	3	3
M3	-	-	**1**	1	1	**3**	3	3	**2**	2	2	2	2	2	2	2	2	**7**	7	7	**3**	3	3	**0**	0	0	**2**	2	2	**1**
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	**F**	**F**	**F**	H	H	**F**	H	**F**	H	**F**	H	H	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**	**F**
Sale				7		1		2	3	0	4			0		3		2			7	0	1	3	5	2	0	1	4	2

Resultado: 22 fallos, 8 aciertos, tasa de fallos 73.33 % y 8 escrituras a disco. Reproduce exactamente la traza de LRU en los 30 pasos.

Código fuente en C: `src/algoritmos/aging.c` (ejecución: `./simulador -a 9`)

```c
/*
 * aging.c - Algoritmo 9: Envejecimiento (Aging)
 * Responsable: Integrante 5
 *
 * Idea (Tanenbaum 3.4.7): aproximación de LRU en software. Cada marco tiene
 * un registro de AGING_BITS bits. Al final de cada tic de reloj:
 *     reg = (reg >> 1) | (R << (AGING_BITS-1));   R = 0
 * Así el bit más significativo indica "usada en el último tic", el siguiente
 * "usada hace 2 tics", etc. El registro más PEQUEÑO es la página más
 * "vieja": esa es la víctima (empate: menor índice de marco).
 * Página recién cargada: registro 0 y R = 1.
 *
 * Variables auxiliares: registros en binario, DESPUÉS del tic.
 */
#include "../algoritmos.h"

/* Escribe los registros como "[10000000,01000000,-]" */
static const char *fmt_registros(char *buf, size_t len, const unsigned reg[],
                                 const int marcos[], int m)
{
    size_t u = (size_t)snprintf(buf, len, "reg=[");
    for (int k = 0; k < m && u < len; k++) {
        if (marcos[k] == MARCO_VACIO) {
            u += (size_t)snprintf(buf + u, len - u, "-");
        } else {
            for (int b = AGING_BITS - 1; b >= 0 && u < len; b--)
                u += (size_t)snprintf(buf + u, len - u, "%c", (reg[k] >> b) & 1u ? '1' : '0');
        }
        if (u < len)
            u += (size_t)snprintf(buf + u, len - u, "%s", (k + 1 < m) ? "," : "]");
    }
    return buf;
}

void simular_aging(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    unsigned reg[MAX_MARCOS] = {0};
    int bit_r[MAX_MARCOS] = {0};
    char buf[AUX_LEN];
    const unsigned mascara = (1u << AGING_BITS) - 1u;

    resultado_iniciar(r, "Aging", cfg);
    marcos_vaciar(marcos, cfg->marcos);
```

```
    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco = -1, victima = -1;

        if (idx >= 0) {
            bit_r[idx] = 1;
        } else {
            marco = buscar_libre(marcos, cfg->marcos);
            if (marco < 0) {
                marco = 0;
                for (int k = 1; k < cfg->marcos; k++)
                    if (reg[k] < reg[marco])
                        marco = k;
                victima = marcos[marco];
            }
            marcos[marco] = pagina;
            reg[marco] = 0;
            bit_r[marco] = 1;
        }

        /* Tic de reloj: desplazar e insertar R por la izquierda */
        for (int k = 0; k < cfg->marcos; k++) {
            if (marcos[k] == MARCO_VACIO)
                continue;
            reg[k] = ((reg[k] >> 1) | ((unsigned)bit_r[k] << (AGING_BITS - 1))) & mascara;
            bit_r[k] = 0;
        }

        registrar_paso(r, cfg, i, marcos, marco, victima, "%s",
                       fmt_registros(buf, sizeof buf, reg, marcos, cfg->marcos));
    }
}
```

2.3.11 Algoritmo 10: Conjunto de Trabajo (Working Set)

Responsable: Integrante 5 – [Nombre y Apellidos del Integrante 5].

Principio de operación. Expulsa una página **fuera del conjunto de trabajo**: con R = 0 y sin usarse en los últimos $\tau = 4$ tics. Cada marco guarda R y el tiempo del último uso (TUU); R se limpia cada 5 tics. **Ventaja:** relaciona el reemplazo con la localidad. **Desventaja:** más costoso; depende de τ. **Anomalía de Belády:** posible con marcos fijos (se halló un caso). (Denning (1968)).

Tabla 12

Traza de Conjunto de Trabajo con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	**3**	3	3	**2**	**3**	3	3	3	**1**	1	1	1	1	1	1	**3**	3	3	**0**	**1**	**4**	4	4	**0**	**1**
M2	-	**0**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**5**	5	5	5	5	5	**3**	3	3
M3	-	-	**1**	1	1	1	1	**4**	4	4	4	4	**2**	2	2	2	2	**7**	7	7	7	7	**2**	2	2	2	2	2	2	2
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	**F**	**F**	H	H	**F**	**F**	H	H	H	**F**	H	H	**F**	**F**	**F**	**F**	**F**	**F**	H	**F**	**F**	**F**
Sale				7		2		1	3	2			4	3				2			1	0	7	3	0	1		5	4	0

Resultado: 20 fallos, 10 aciertos, tasa de fallos 66.67 % y 7 escrituras a disco. En el paso 23 la página 7 tenía edad $5 > \tau$ y salió.

Código fuente en C: `src/algoritmos/working_set.c` (ejecución: `./simulador -a 10`)

```c
/*
 * working_set.c - Algoritmo 10: Working Set (Conjunto de Trabajo)
 * Responsable: Integrante 5
 *
 * Idea (Denning 1968; versión de reemplazo de Tanenbaum 3.4.8):
 * el conjunto de trabajo son las páginas usadas en los últimos WS_TAU tics.
 * Una página FUERA del conjunto de trabajo es buena víctima.
 *
 * Cada marco guarda R y TUU (tiempo virtual del último uso). El TUU NO se
 * actualiza en cada acierto (eso sería LRU exacto, muy caro): se actualiza
 * al cargar y durante el barrido en un fallo. Los bits R se limpian con
 * una interrupción de reloj cada PERIODO_RELOJ tics.
 *
 * Fallo con memoria llena, barrido de M1 a Mn (t = paso actual, base 1):
 *   - R = 1            -> TUU = t (está en el conjunto de trabajo).
 *   - R = 0, edad > τ  -> fuera del conjunto: la PRIMERA encontrada es la
 *                         víctima (el barrido sigue solo para actualizar TUU).
 *   - R = 0, edad <= τ -> se recuerda la de mayor edad.
 *   Sin víctima: la de mayor edad con R = 0; si todas tienen R = 1, M1.
 *
 * Variables auxiliares: bits R, TUU de cada marco y aviso de reloj.
 */
#include "../algoritmos.h"

void simular_working_set(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int tuu[MAX_MARCOS];
    char br[AUX_LEN / 3], bt[AUX_LEN / 3];

    resultado_iniciar(r, "Working Set", cfg);
    marcos_vaciar(marcos, cfg->marcos);
    marcos_vaciar(tuu, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int t = i + 1;
        int pagina = cfg->refs[i];
        int reset = (i > 0 && i % PERIODO_RELOJ == 0);
        int idx, marco, victima = -1;
        const char *motivo = "";

        if (reset)
            for (int k = 0; k < cfg->marcos; k++)
                bit_r[k] = 0;

        idx = buscar_pagina(marcos, cfg->marcos, pagina);
        if (idx >= 0) {
            bit_r[idx] = 1;
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s TUU=%s t=%d%s",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), t,
                           reset ? " [reloj: R<-0]" : "");
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            int fuera = -1, mas_viejo = -1, edad_max = -1;
            for (int k = 0; k < cfg->marcos; k++) {
                if (bit_r[k] == 1) {
                    tuu[k] = t;
                } else {
                    int edad = t - tuu[k];
                    if (edad > WS_TAU && fuera < 0)
                        fuera = k;
                    if (edad > edad_max) {
                        edad_max = edad;
                        mas_viejo = k;
                    }
                }
```

```
            }
            if (fuera >= 0) {
                marco = fuera;
                motivo = " fuera del WS";
            } else if (mas_viejo >= 0) {
                marco = mas_viejo;
                motivo = " todas en WS: la mas vieja";
            } else {
                marco = 0;
                motivo = " todas R=1: M1";
            }
            victima = marcos[marco];
        }
        marcos[marco] = pagina;
        bit_r[marco] = 1;
        tuu[marco] = t;

        registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s TUU=%s t=%d%s%s",
                       fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                       fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), t,
                       reset ? " [reloj: R<-0]" : "", motivo);
    }
}
```

2.3.12 Algoritmo 11: WSClock (Working Set Clock)

Responsable: Integrante 6 – [Nombre y Apellidos del Integrante 6].

Principio de operación. Reloj combinado con el conjunto de trabajo: expulsa una página **vieja y limpia**; si está vieja y sucia, **programa su escritura** y sigue buscando. **Ventaja:** eficiente y con pocas escrituras. **Desventaja:** más complejo que el Reloj. **Anomalía de Belády: sí** (9 → 10). (Carr y Hennessy (1981)).

Tabla 13

Traza de WSClock con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	**4**	4	**3**	3	3	3	**1**	1	1	1	**7**	7	**1**	**3**	3	3	3	3	3	**2**	2	2	**1**
M2	-	**0**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**5**	5	**0**	0	0	0	0	0	0
M3	-	-	**1**	1	1	**3**	3	3	**2**	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	**1**	**4**	4	**3**	3	3
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	**F**	**F**	H	H	H	**F**	H	H	H	**F**	H	**F**	**F**	**F**	H	**F**	**F**	**F**	**F**	**F**	H	**F**
Sale				7		1		2	3	4				3				1		7	1	0		5	2	1	3	4		2

Resultado: 19 fallos, 11 aciertos, tasa de fallos 63.33 % y 4 escrituras a disco. Junto con LFU, es el mejor algoritmo realizable (19 fallos, 4 escrituras).

Código fuente en C: `src/algoritmos/wsclock.c` (ejecución: `./simulador -a 11`)

```
/*
 * wsclock.c - Algoritmo 11: WSClock (Working Set Clock)
 * Responsable: Integrante 6
 *
 * Idea (Carr y Hennessy 1981; Tanenbaum 3.4.9): combina Reloj y conjunto de
 * trabajo. Cada marco guarda R, M y TUU (tiempo del último uso).
 * Acierto: R = 1, TUU = t, M |= W.
 * Fallo con memoria llena, el puntero recorre como máximo una vuelta:
 *   - R = 1                    -> R = 0, TUU = t, avanza.
 *   - R = 0, edad > τ, M = 0   -> VÍCTIMA (vieja y limpia).
```

```
 *   - R = 0, edad > τ, M = 1   -> se programa su escritura a disco
 *                                 (M = 0, cuenta 1 escritura) y avanza.
 *   - R = 0, edad <= τ         -> está en el conjunto de trabajo, avanza.
 * Si la vuelta termina sin víctima: el primer marco (desde el puntero) con
 * R = 0 y M = 0; si no existe, el marco bajo el puntero.
 *
 * Variables auxiliares: bits R, M, TUU, puntero y escrituras programadas.
 */
#include "../algoritmos.h"

void simular_wsclock(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int bit_m[MAX_MARCOS] = {0};
    int tuu[MAX_MARCOS];
    int puntero = 0;
    char br[AUX_LEN / 4], bm[AUX_LEN / 4], bt[AUX_LEN / 4];

    resultado_iniciar(r, "WSClock", cfg);
    marcos_vaciar(marcos, cfg->marcos);
    marcos_vaciar(tuu, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int t = i + 1;
        int pagina = cfg->refs[i];
        int w = es_escritura(cfg, i);
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1, programadas = 0;

        if (idx >= 0) {
            bit_r[idx] = 1;
            bit_m[idx] |= w;
            tuu[idx] = t;
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s M=%s TUU=%s ptr=M%d",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), puntero + 1);
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = -1;
            for (int k = 0; k < cfg->marcos && marco < 0; k++) {
                int c = puntero;
                if (bit_r[c] == 1) {
                    bit_r[c] = 0;
                    tuu[c] = t;
                } else if (t - tuu[c] > WSCLOCK_TAU) {
                    if (bit_m[c] == 0) {
                        marco = c;                  /* vieja y limpia */
                        break;
                    }
                    bit_m[c] = 0;                   /* programar escritura */
                    marcar_escrito(r, c);
                    programadas++;
                }
                puntero = (puntero + 1) % cfg->marcos;
            }
            if (marco < 0) {
                for (int k = 0; k < cfg->marcos && marco < 0; k++) {
                    int c = (puntero + k) % cfg->marcos;
                    if (bit_r[c] == 0 && bit_m[c] == 0)
                        marco = c;
                }
                if (marco < 0)
                    marco = puntero;
            }
            victima = marcos[marco];
            puntero = (marco + 1) % cfg->marcos;
        }
        marcos[marco] = pagina;
```

```
        bit_r[marco] = 1;
        bit_m[marco] = w;
        tuu[marco] = t;

        if (programadas > 0)
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s TUU=%s ptr=M%d escr.prog=%d",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), puntero + 1, programadas);
        else
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s TUU=%s ptr=M%d",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), puntero + 1);
    }
}
```

2.3.13 Algoritmo 12: Reemplazo Aleatorio (Random)

Responsable: Integrante 6 – [Nombre y Apellidos del Integrante 6].

Principio de operación. Elige la víctima al azar con un generador congruencial lineal propio, x = (1103515245·x + 12345) mod 2^{31}, semilla 42, para que el resultado sea reproducible. **Ventaja:** costo mínimo. **Desventaja:** resultado variable según la semilla. **Anomalía de Belády: sí** (posible en cualquier caso). (Belady (1966)).

Tabla 14

Traza de Reemplazo Aleatorio con 3 marcos y 30 referencias

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	**7**	7	7	**2**	2	2	2	2	2	2	2	**3**	3	**1**	1	1	1	1	1	1	1	1	**2**	2	2	2	2	**3**	3	3
M2	-	**0**	0	0	0	0	0	0	0	**3**	**0**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**4**	4	4	4	4
M3	-	-	**1**	1	1	**3**	3	**4**	4	4	4	4	**2**	2	2	2	2	**7**	7	7	**3**	**5**	5	5	**1**	1	1	1	**0**	**1**
Res	**F**	**F**	**F**	**F**	H	**F**	H	**F**	H	**F**	**F**	**F**	**F**	**F**	H	H	H	**F**	H	H	**F**	**F**	**F**	H	**F**	**F**	H	**F**	**F**	**F**
Sale				7		1		3		0	3	2	4	3				2			7	3	1		5	0		2	1	0

Resultado: 20 fallos, 10 aciertos, tasa de fallos 66.67 % y 8 escrituras a disco. Con la semilla 42 obtiene 20 fallos; con otra semilla el resultado cambia.

Código fuente en C: `src/algoritmos/aleatorio.c` (ejecución: `./simulador -a 12`)

```
/*
 * aleatorio.c - Algoritmo 12: Aleatorio (Random Replacement, RAND)
 * Responsable: Integrante 6
 *
 * Idea: la víctima se elige al azar entre los marcos ocupados. No usa
 * ninguna información de uso; sirve como referencia de "sin inteligencia".
 *
 * Para que el resultado sea REPRODUCIBLE en cualquier compilador se usa un
 * generador congruencial lineal (LCG) propio en lugar de rand():
 *     x = (1103515245 * x + 12345) mod 2^31,   semilla x0 = RANDOM_SEMILLA
 *     víctima = x mod n_marcos
 * El generador solo avanza cuando hay que reemplazar.
 *
 * Variables auxiliares: valor x generado y marco elegido.
 */
#include <stdint.h>
#include "../algoritmos.h"
```

```
static uint32_t lcg_siguiente(uint32_t *x)
{
    *x = (1103515245u * *x + 12345u) & 0x7FFFFFFFu;   /* mod 2^31 */
    return *x;
}

void simular_aleatorio(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    uint32_t x = RANDOM_SEMILLA;

    resultado_iniciar(r, "Aleatorio (RAND)", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int marco, victima = -1;

        if (buscar_pagina(marcos, cfg->marcos, pagina) >= 0) {
            registrar_paso(r, cfg, i, marcos, -1, -1, "x=%lu", (unsigned long)x);
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            uint32_t v = lcg_siguiente(&x);
            marco = (int)(v % (uint32_t)cfg->marcos);
            victima = marcos[marco];
            marcos[marco] = pagina;
            registrar_paso(r, cfg, i, marcos, marco, victima, "x=%lu -> x mod %d = %d (M%d)",
                           (unsigned long)v, cfg->marcos, marco, marco + 1);
            continue;
        }
        marcos[marco] = pagina;
        registrar_paso(r, cfg, i, marcos, marco, victima, "x=%lu", (unsigned long)x);
    }
}
```

2.4 Comparación de resultados

Tabla 15

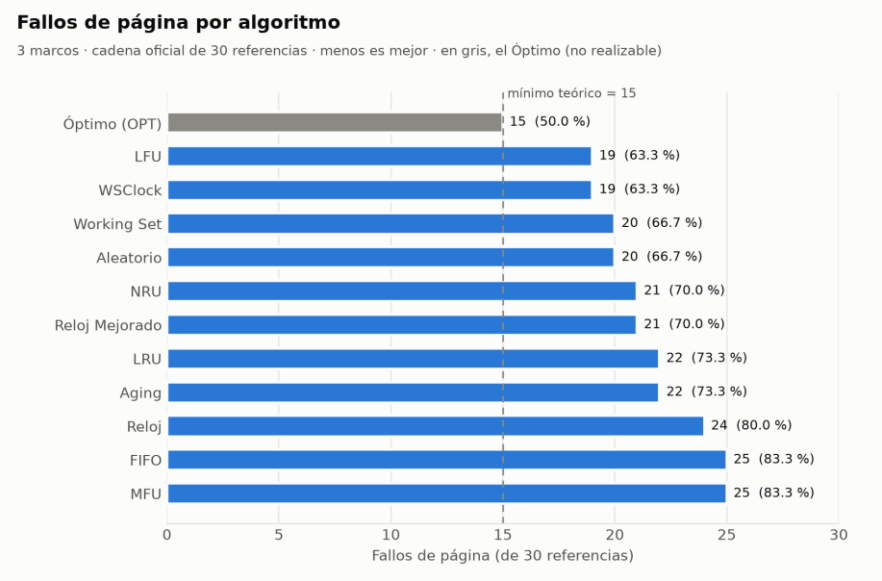
Resultados de los 12 algoritmos con 3 marcos y 30 referencias

#	Algoritmo	Fallos	Aciertos	% Fallos	Escrituras
1	FIFO	25	5	83.33 %	9
2	Óptimo	15	15	50.00 %	6
3	LRU	22	8	73.33 %	8
4	Reloj o Segunda Oportunidad	24	6	80.00 %	9
5	NRU	21	9	70.00 %	7
6	LFU	19	11	63.33 %	4
7	MFU	25	5	83.33 %	9
8	Reloj Mejorado	21	9	70.00 %	6
9	Envejecimiento	22	8	73.33 %	8
10	Conjunto de Trabajo	20	10	66.67 %	7
11	WSClock	19	11	63.33 %	4
12	Reemplazo Aleatorio	20	10	66.67 %	8

Nota. Elaboración propia con el simulador del proyecto.

Figura 1

Fallos de página por algoritmo (3 marcos)

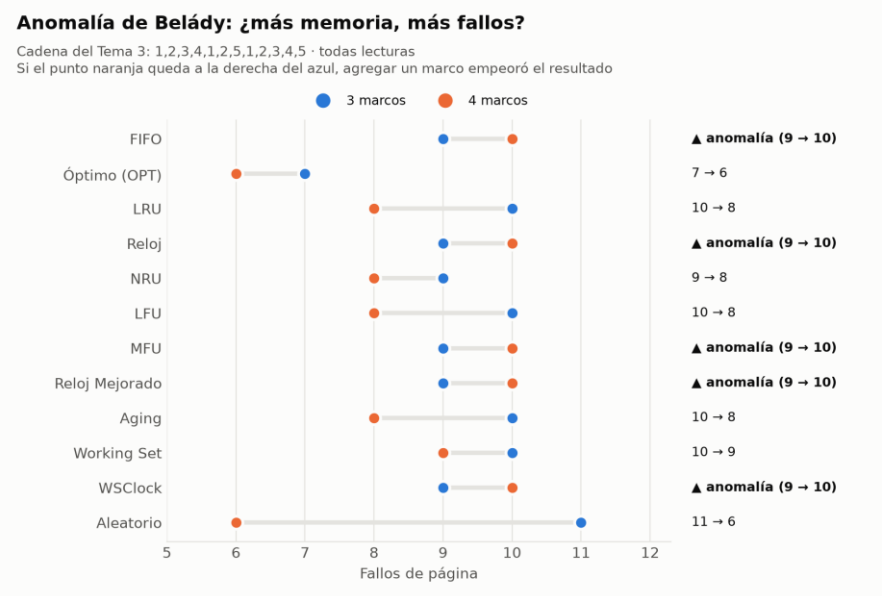


Nota. Elaboración propia con los datos del simulador en C.

El Óptimo marca el mínimo (15). LFU y WSClock son los mejores algoritmos realizables (19) y FIFO y MFU los peores (25).

Figura 2

Anomalía de Belády con la cadena 1,2,3,4,1,2,5,1,2,3,4,5



Nota. Elaboración propia con los datos del simulador en C.

Cinco algoritmos empeoran al pasar de 3 a 4 marcos; LRU y el Óptimo, algoritmos de pila, no.

CONCLUSIONES

- Los 12 algoritmos se implementaron en C para Linux en un simulador modular que compila sin advertencias y muestra la traza de cada uno.
- Con 3 marcos y 30 referencias, el **Óptimo** obtuvo 15 fallos (el mínimo); **LFU y WSClock**, 19 (los mejores realizables, con solo 4 escrituras); y **FIFO y MFU**, 25 (los peores).
- **Reloj Mejorado** igualó a **NRU** en fallos (21) pero escribió menos, y **Aging** reprodujo exactamente a **LRU** (22).
- Se verificó la **anomalía de Belády**: FIFO, Reloj, MFU, Reloj Mejorado y WSClock pasan de 9 a 10 fallos con un marco más; LRU y el Óptimo, algoritmos de pila, nunca la presentaron.

RECOMENDACIONES

- Repetir el experimento con cadenas de programas reales y con varios procesos.
- Ejecutar `make validar` antes de modificar el simulador.

BIBLIOGRAFÍA

Belady, L. A. (1966). A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal, 5*(2), 78–101. https://doi.org/10.1147/sj.52.0078

Belady, L. A., Nelson, R. A., & Shedler, G. S. (1969). An anomaly in space-time characteristics of certain programs running in a paging machine. *Communications of the ACM, 12*(6), 349–353. https://doi.org/10.1145/363011.363155

Carr, R. W., & Hennessy, J. L. (1981). WSCLOCK—A simple and effective algorithm for virtual memory management. En *Proceedings of the Eighth ACM Symposium on Operating Systems Principles* (pp. 87–95). ACM. https://doi.org/10.1145/800216.806596

Corbató, F. J. (1969). A paging experiment with the Multics system. En H. Feshbach & K. U. Ingard (Eds.), *In honor of Philip M. Morse* (pp. 217–228). MIT Press.

Denning, P. J. (1968). The working set model for program behavior. *Communications of the ACM, 11*(5), 323–333. https://doi.org/10.1145/363095.363141

Kilburn, T., Edwards, D. B. G., Lanigan, M. J., & Sumner, F. H. (1962). One-level storage system. *IRE Transactions on Electronic Computers, EC-11*(2), 223–235. https://doi.org/10.1109/TEC.1962.5219356

Ruiz L., E., & Raffo L., E. (2003). Simulación de la memoria cache. *Industrial Data, 6*(2), 48–55. https://www.redalyc.org/articulo.oa?id=81660207

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.

ANEXO: COMPILACIÓN Y EJECUCIÓN EN LINUX

Los 12 módulos se compilan junto con los archivos comunes del proyecto (`parametros`, `comun`, `algoritmos`, `main.c` y `Makefile`), disponibles en el repositorio:

```
$ cd src
$ make                    # gcc -std=c11 -Wall -Wextra -pedantic -O2 (0 advertencias)
$ ../build/simulador      # traza de los 12 algoritmos + tabla comparativa
$ ../build/simulador -a 2 # solo un algoritmo (1..12)
$ make validar            # 67 pruebas automaticas
```